

# ECE 545 - Cryptographic Engineering and Hardware Security

## Assignment 2: Breaking AES through Differential Power Analysis

Dr. Aydin Aysu  
Email: aaysu@ncsu.edu

September 4, 2026

### 1 Introduction and Goals

The purpose of this assignment is to provide a hands-on introduction to power-based side-channel analysis. Since you do not have access to hardware, you will be using a Python-based environment to extract power traces from a Verilog simulation during AES-ECB executions. With the known associated input/output values of AES during the simulation, you will be asked to extract the 128-bit secret key via Differential Power Analysis (DPA).

The assignment is provided to you with one Jupyter Notebook file in addition to this .pdf instructions file, named `DPA_Assignment.ipynb`. At a high level, the Jupyter Notebook will download the required files for running the AES DPA attack, perform iVerilog simulation of AES using known plaintexts (either 5000 or 10000), then extract and interpolate the power trace from the simulation's Value Change Dump (VCD). You will then be provided with the plaintexts, the corresponding ciphertexts, and the power trace associated with these AES operations. You will be asked to extract the AES secret key from this. A detailed explanation of the process is in the Jupyter Notebook. **Please read it carefully.**

#### 1.1 Graded Items

You will turn in a soft copy report of your results as well as any code to the TA on Moodle. There are three tasks, each with multiple subtasks in the Jupyter Notebook. These tasks also include nine questions and four figures. Make sure to answer all questions in your report. Make sure to include the following figures, as mentioned in the Jupyter Notebook.

- **Figure 1:** Plot of the first power trace.
- **Figure 2:** Plot for all key guesses.
- **Figure 3:** Plot of two best key guesses.
- **Figure 4:** Plot the “evolution” of all key hypotheses for 10000 measurements at this particular time instance.

Each question is worth 12.5 points. The last question, therefore, can be treated as bonus. Lab reports must be in readable English and not raw dumps of log files. Your lab reports must be typeset and must not exceed 6 pages. You will be required to use LaTeX to generate the reports. You can use [Overleaf](#) to generate the LaTeX file. Please submit your lab report (in .pdf format) and all of your code in a .zip file on [Moodle](#) as `lab2-YOUR.UNITY.ID.HERE.zip`

You can use any software language you prefer to write the programs required to complete this assignment. However, it is preferable to use Python directly in the Google Colab environment, as using something else would require you to either extract the power trace from the CSV file yourself or dump the power trace output that is generated by Google Colab Jupyter Notebook into your IDE. If you have any questions about the assignment, please first refer to the TA of the course (Sharath Pendyala, [spendya@ncsu.edu](mailto:spendya@ncsu.edu)).

## 2 Google Colab Python Environment

Google Colaboratory, commonly known as Google Colab, is a free, cloud-based Jupyter Notebook environment hosted by Google. It allows users to write, execute, and share Python code directly in a web browser without requiring any local software installation or package configuration. Colab is widely used for data science, machine learning, and computational tasks because it comes pre-installed with many popular data analysis and visualization libraries (such as NumPy, Pandas, and Matplotlib). For this assignment, it serves as an accessible platform to run iVerilog simulation, process the CSV trace file to generate power traces, process the power trace into useful power trace plots for all input plaintext-ciphertext combinations, implement DPA techniques on the extracted power trace, and plot your visual findings seamlessly.

The primary advantages of Google Colab include its zero-configuration setup, platform independence, and free access to computing resources, which allows students to execute demanding scripts regardless of their local computer's processing power. Furthermore, it integrates directly with Google Drive, making it simple to save work and collaborate. However, there are a few disadvantages to consider. Because it is a cloud service, it requires a continuous internet connection. Furthermore, sessions are temporary and time-limited; if a notebook is left idle for too long, the environment will disconnect, and any locally uploaded files (such as bulky power traces) or variables in memory will be lost and must be reloaded when the session restarts.

## 3 How to use Google Colab

1. Open Google Colab.
  - (a) Open the website: <https://colab.research.google.com/>
  - (b) Sign in using your NCSU email address.
2. Upload the Jupyter notebook.
  - (a) Download the Jupyter Notebook `DPA.Assignment.ipynb` on to your computer.
  - (b) On Google Colab, select the “Upload Notebook” option to upload `NAME_OF_FILE.ipynb`.

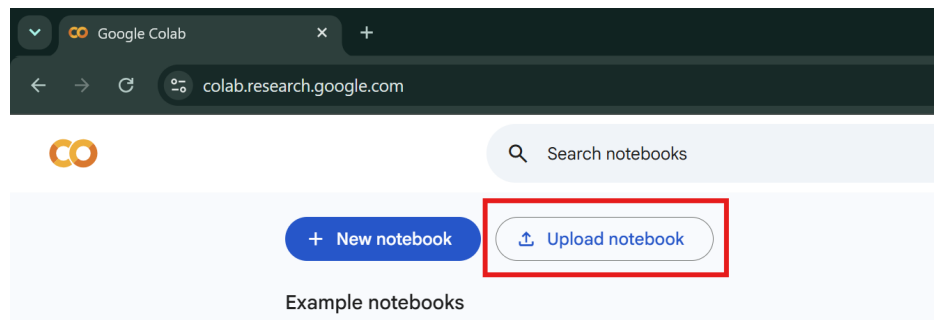


Figure 1: Uploading the Jupyter Notebook to the Google Colab environment.

- (c) You should now be able to view the entire notebook as seen in Figure 2. You can browse individual sections using the table of contents option on the left side.
  - (d) The uploaded file gets saved to your NCSU Google Drive. Please note that the Jupyter notebook file on your PC will no longer be used or needed, so delete it, as you might get confused between the file on your PC and the file on Google Colab (your NCSU Google Drive)
3. Executing the Code
  - (a) Jupyter Notebooks have two types of cells: Text Cell and Code Cell.

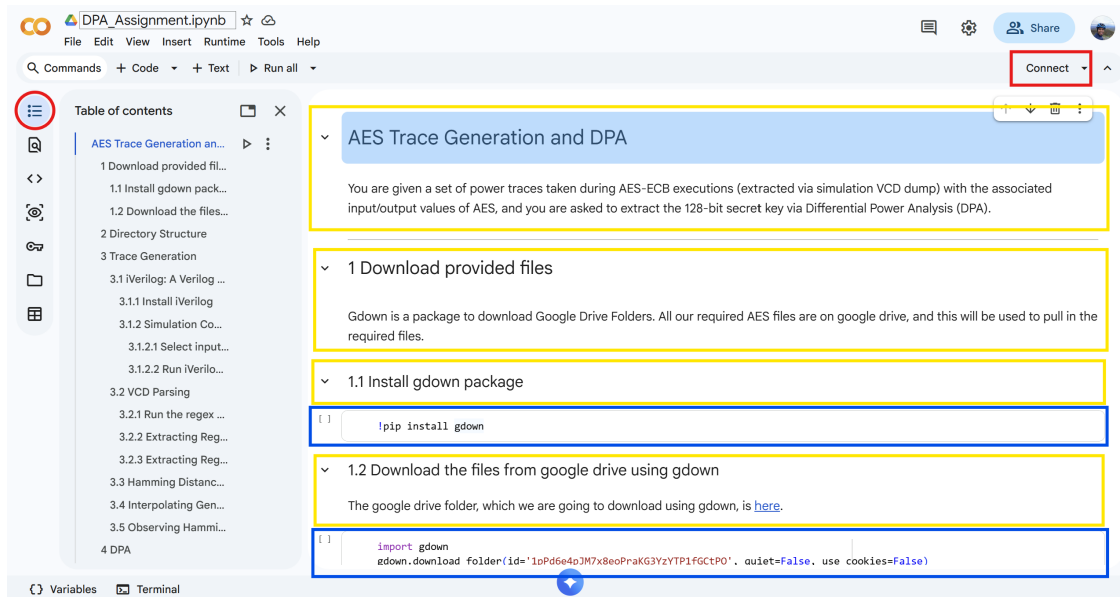


Figure 2: Browsing the Jupyter Notebook on Google Colab.

- (b) Text Cells are for writing a description, or some sort of comment, or an explanation, whereas Code Cells are for writing your Python code. The yellow boxes in Figure 2 are Text Cells and the blue boxes are Code Cells.
- (c) You can add cells between any two cells in the notebook by mousing over the gap between the two cells, as seen in Figure 3 blue and yellow boxes. You can remove cells you don't need (red box), or move them to a different location in the notebook (green box).

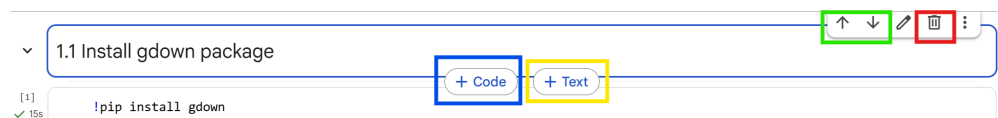


Figure 3: Add, move, or delete cells on Google Colab.

- (d) Since your code runs on Google's servers, your connection status is shown in the top right corner, as seen in Figure 2. Your Google Colab environment might disconnect after a certain period of inactivity, but your code is not lost. However, when you reconnect, you might have to run the code cells from the very beginning (or as needed), as the data is not persistent on the servers between sessions. You also lose any files you might have created or uploaded to the session.
- (e) Each Code Cell can be run by clicking the "Play" button when you mouse over that cell, or by selecting the Code Cell you want to run and pressing **Ctrl + Enter**, or pressing **Shift + Enter**, which executes cells sequentially, one per press. An example is shown in Figure 4. Please note that these are for a Windows PC. On a MAC, you will have to use **CMD** instead of **Ctrl**.

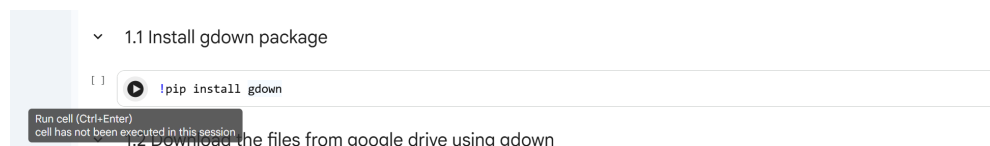


Figure 4: Running Code Cells on Google Colab.

- (f) During an ongoing session on Google Colab, the data and any related functions from all executed Code Cells are persistent and in memory. Future cells can reuse any data or code produced by cells that were previously executed. This data, however, is lost when you lose connection to the session (disconnect due to inactivity or close the browser). You will also lose the simulation dump files when this happens.
- (g) If you want to save time with RTL simulations on Google Colab, I would suggest that you download the outputs generated during the iVerilog simulation (Section 3.1.2.2 in the Jupyter Notebook), save it somewhere on your computer, and upload it back into the Google Colab environment (make sure to place it in the right directory location) when you want to resume your assignment work.
- (h) Google Colab is a lot more powerful than what I've explained here, as I have only scratched the surface here. You can read up on its other functions on Google Colab blog or other sources.